



KubeCon



CloudNativeCon

Europe 2026

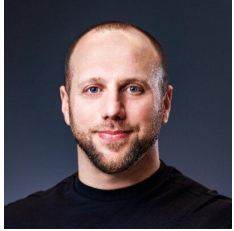
#KubeCon #CloudNativeCon

Make GenAI Production-Ready with Kubernetes Patterns

Bilgin Ibryam (Diagrid) & Roland Huß (Red Hat)



We Wrote the Patterns. Now We Apply Them to GenAI.



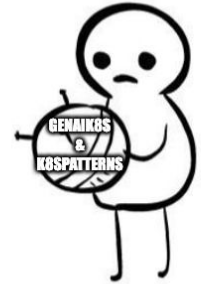
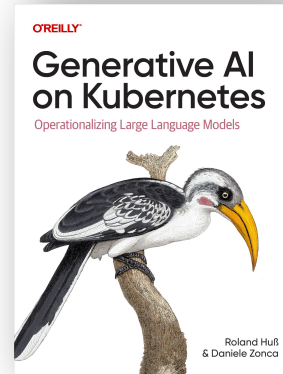
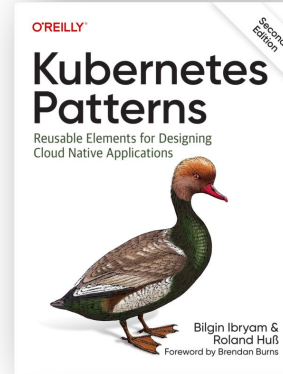
Bilgin Ibryam

Principal Product Manager,
Diagrid



Roland Huß

Distinguished Engineer,
Red Hat



A Kubernetes App You'd Recognize



KubeCon

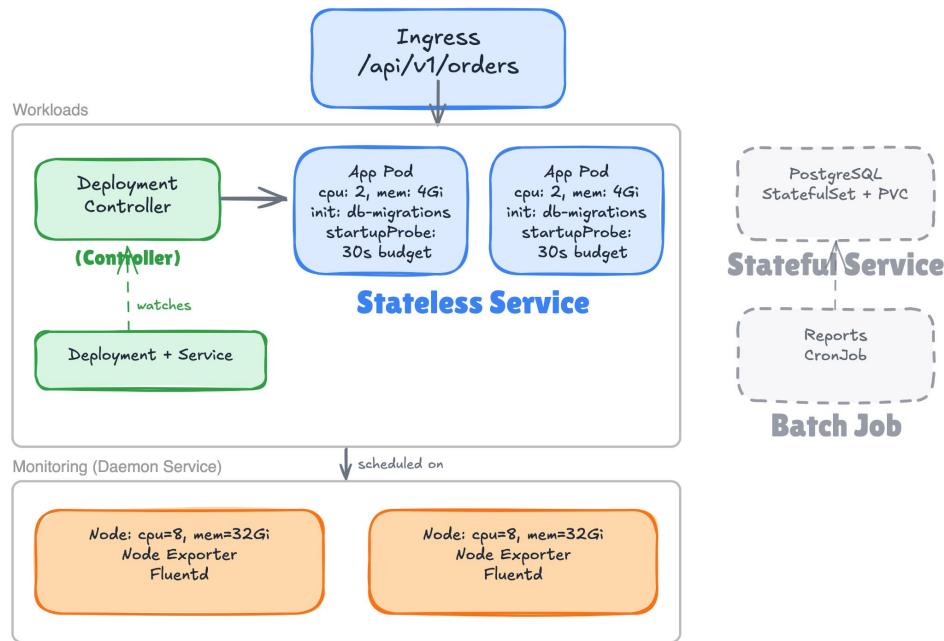


CloudNativeCon

Europe 2026

Kubernetes Patterns

- **Controller**: reconcile desired state, continuously
- **Stateless Service**: identical, replaceable replicas
- **Init Container**: fetch data before the app starts
- **Stateful Service**: stable identity + persistent storage
- **Batch Job**: run to completion, then stop
- **Daemon Service**: one agent per node



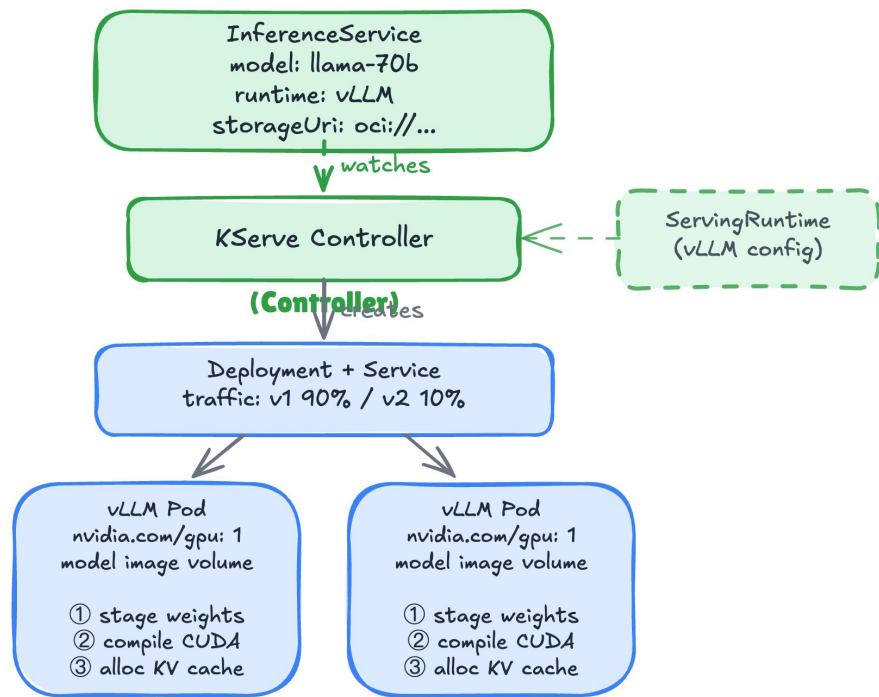
The Inference Service Is Not a Web App

The model workload:

- Tens to hundreds of GB, read-only (Llama 3.1 70B FP16, ~141 GB)
- Requires GPU hardware (VRAM)
- Minutes to load, seconds to respond

KServe manages the lifecycle:

- **InferenceService**: declare model + runtime
- **ServingRuntime**: pluggable server (vLLM / TGI)
- **LLMInferenceService**: for large-scale LLM deployments



Stateless Service



KubeCon



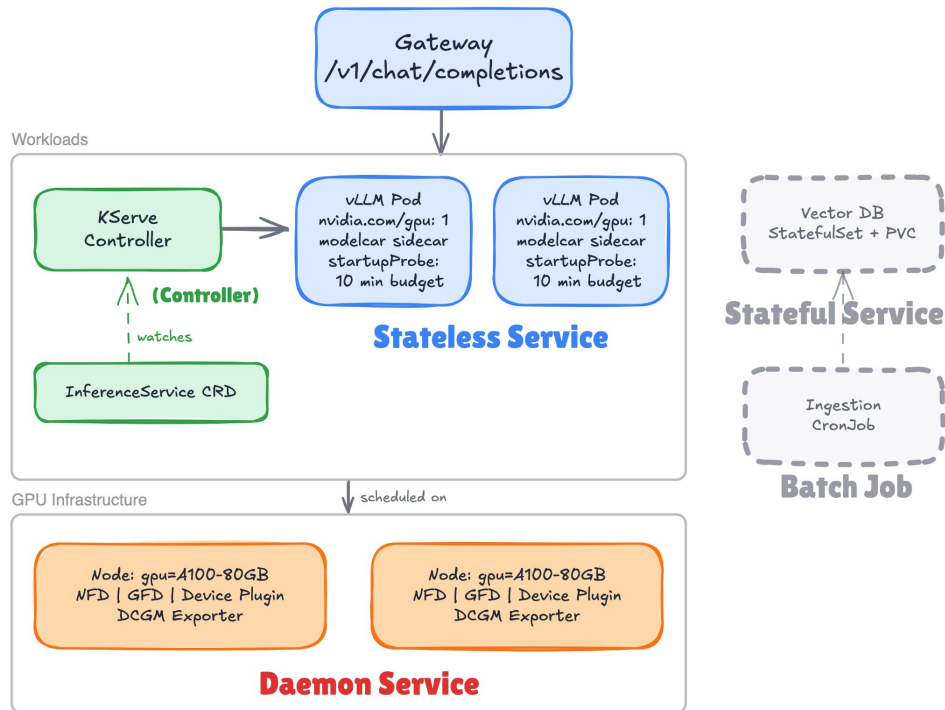
CloudNativeCon

Europe 2026

From App Workload to LLM Workload

Kubernetes Patterns

- **Controller:** KServe: model lifecycle operator
- **Stateless Service:** vLLM: nvidia.com/gpu, modelcar, 10-min startup
- **Init Container:** Downloads 54 GB of weights
- **Stateful Service:** Vector DB: embeddings instead of rows
- **Batch Job:** Ingestion: parse, chunk, embed documents
- **Daemon Service:** GPU stack: NFD, GFD, device plugin, DCGM



KubeCon



CloudNativeCon

Europe 2026

Patterns That Map Directly

K8s Pattern	GenAI Application	What Changes
Controller	KServe — InferenceService CRD	Same reconciliation loop, new CRD
Stateless Service	vLLM / TGI inference server	Weights are read-only, any replica serves any request
Stateful Service	Vector DB (Qdrant, Milvus)	StatefulSet + PVC, stable identity, ordered scaling
Daemon Service	GPU stack (NFD, GFD, device plugin)	One agent per node, infrastructure layer
Predictable Demands	Request: nvidia.com/gpu: 1 Node selector/affinity: gpu.memory > 4gb Dynamic Resource Allocation (DRA)	Declare infrastructure requirements more precisely
Automated Placement	Taints and tolerations	Help scheduler protect your GPU from arbitrary workloads
Adapter	Dapr / Llama Stack as universal AI API	Same companion container, new responsibilities

Familiar Patterns, Unfamiliar Scale

K8s Pattern	GenAI Application	What Changes
Init Container	Loading model data	MB of config → 800 GB of weights
Immutable Configuration	Model as OCI artifact	TB-scale versioned, immutable weights
Health Probe	Model readiness	30s startup → 10-min budget
Declarative Deployment	Model rollouts	Drain window: seconds → 30s+ per in-flight request Canary takes 10 min to load; Traffic split between model versions can be costly
Batch Job	Fine-tuning	Gang scheduling, all-or-nothing start

New GenAI Patterns

- **Model Data Staging** — get heavyweight model data to the Pod fast
- **Token-Aware Routing** — route by model pressure, not request volume
- **RAG Composition** — Higher-level compound pattern, composed of smaller basic patterns.

Pattern: Model Data Initialization

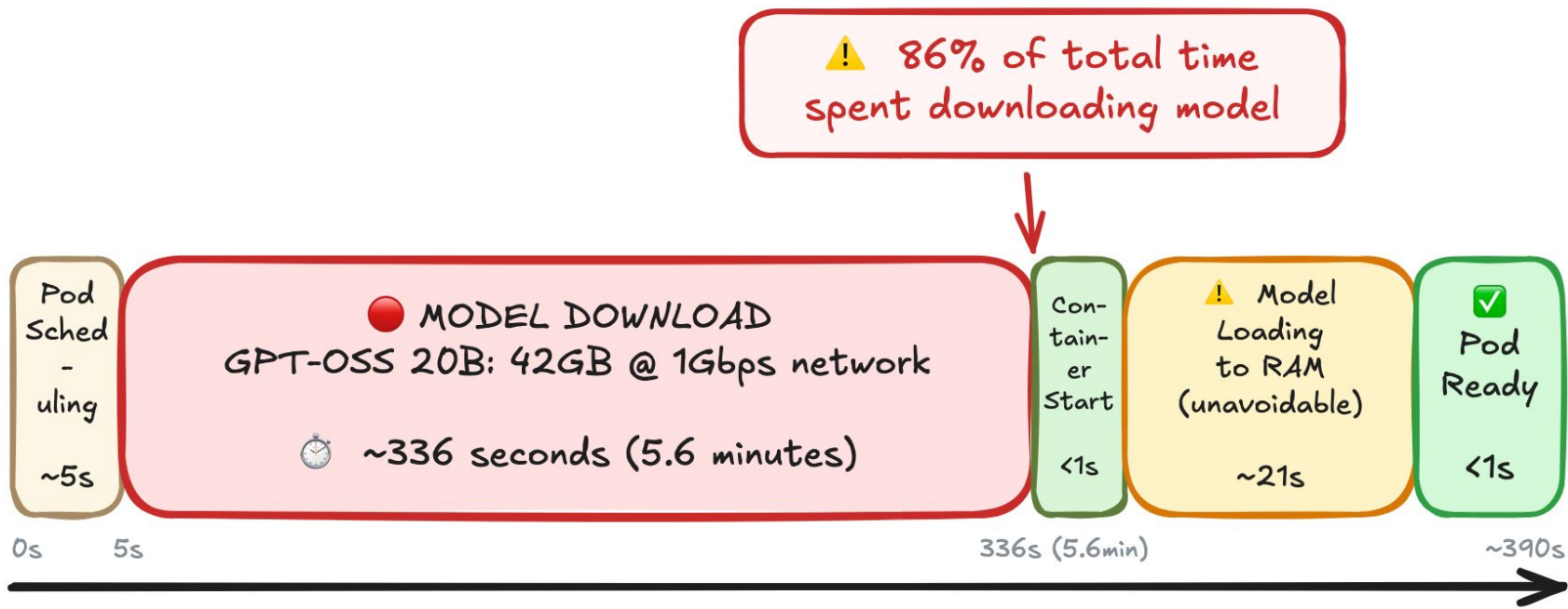


KubeCon



CloudNativeCon

Europe 2026



Total Cold Start Time: ~6.5 minutes

Init Container Copies Model

spec:

initContainers:

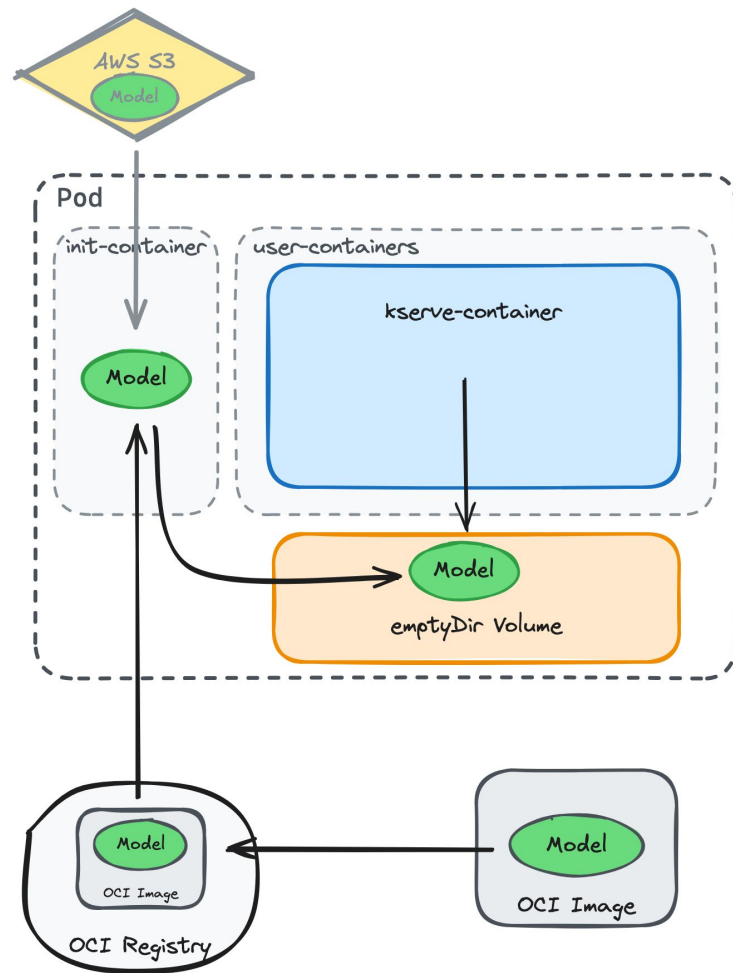
- name: model-loader
- image: registry.io/model:tag
- command:
 - cp -r /models/ /mnt/models/

Strengths:

- Works on any K8s version
- Simple and well-understood

Limitations:

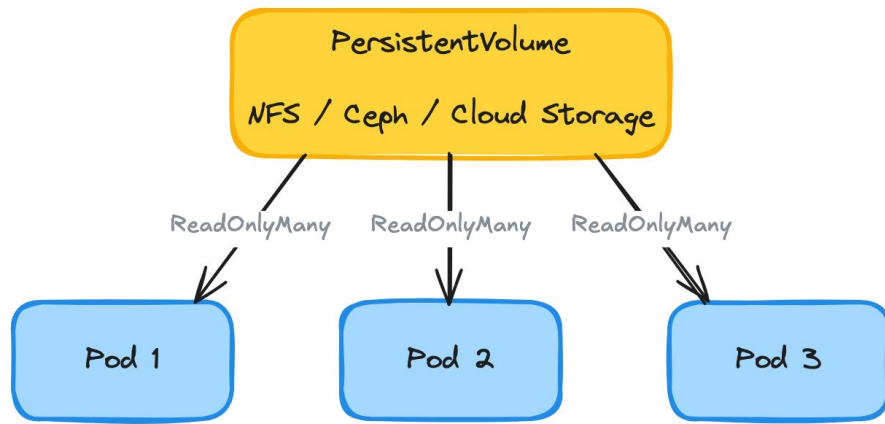
- Full copy every Pod start
- No sharing between Pods



PersistentVolumes Share Models Across Pods

```
kind: PersistentVolumeClaim
spec:
  accessModes: [ReadOnlyMany]
  resources:
    requests:
      storage: 20Gi
```

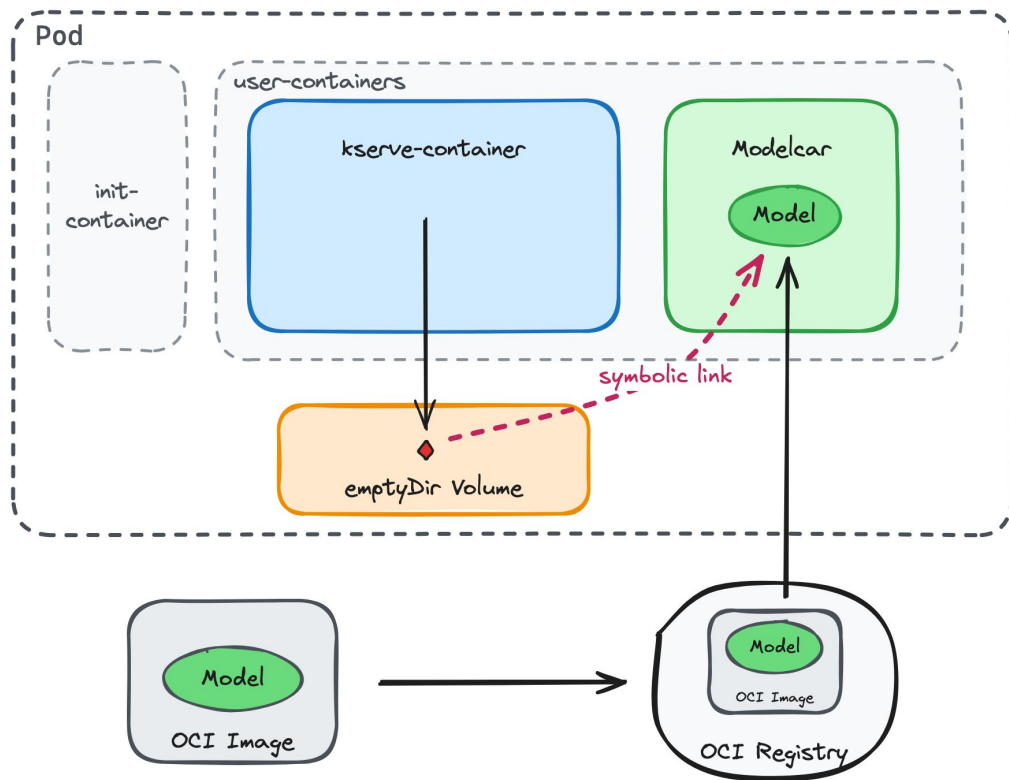
```
# KServe integration
storageUri: pvc://llama-3-8b-pvc/model
```



- Single copy serves all replicas.
- Most common production approach today.

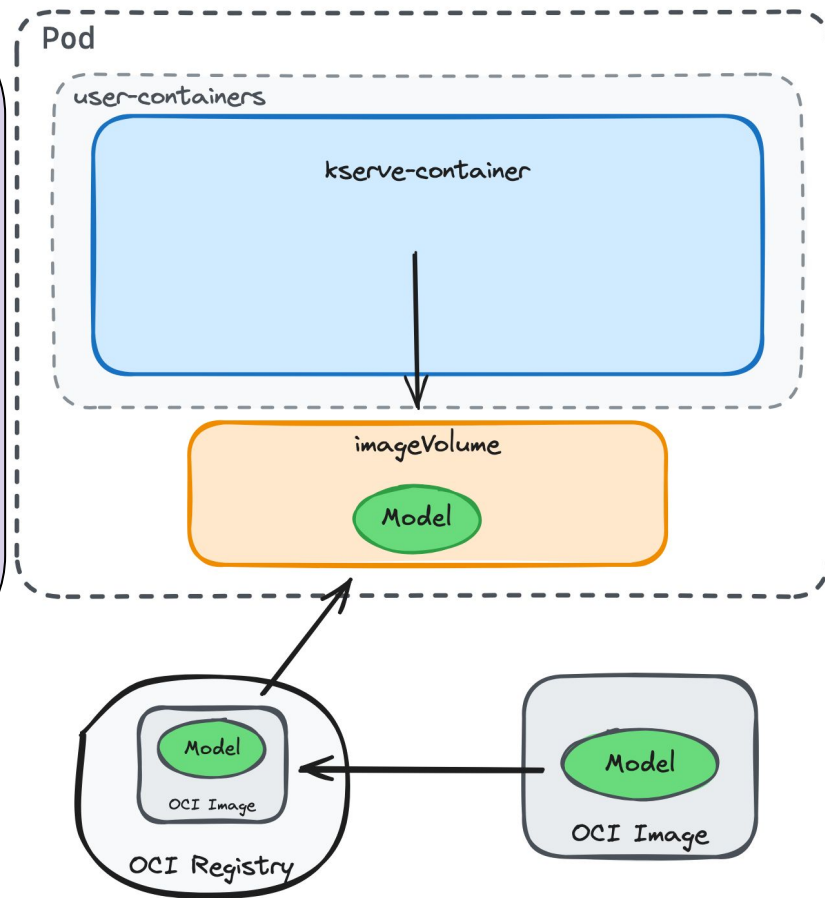
Modelcar Accesses Image Data Via Symlinks

- Model packaged as OCI image
 - Runs as a sidecar container
 - `shareProcessNamespace: true`
 - Symlinks to `/proc/<pid>/root/model`
 - No copy, runtime cache kicks in
-
- KServe follows symlink.
 - First pull ~2 min, then seconds.



ImageVolume Mounts OCI Images Natively

```
spec:
  volumes:
  - name: model-data
    image:
      reference: registry.io/llama-7b:v1
      pullPolicy: IfNotPresent
  containers:
  - name: inference
    volumeMounts:
    - name: model-data
      mountPath: /mnt/models
```



- Alpha 1.31: Aug 2024
- Beta 1.33: Apr 2025
- Beta 1.35: default, Dec 2025
- GA: not yet announced

Strategy Choice Balances Speed and Complexity

Strategy	Speed	Complexity	K8s	Use When
Init Container	Slow	Simple	Any	Small models, simplicity first
PersistentVolume	Fast	Medium	Any	Multiple replicas, shared infra
Modelcar	Fast	Complex	< 1.35	KServe + speed today
ImageVolume	Fast	Simple	1.35+	Future-proof default

ImageVolume is the endgame.

Pattern: Token Aware Routing



KubeCon

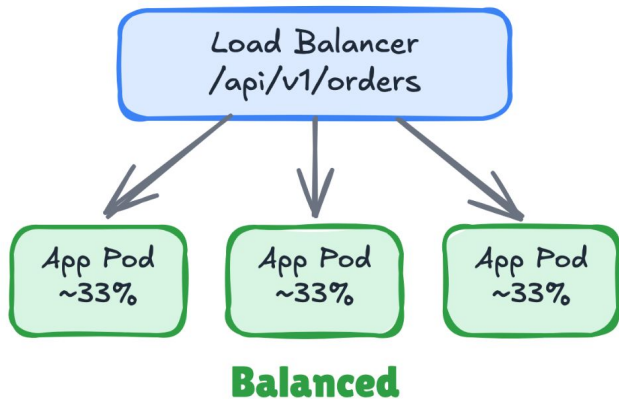


CloudNativeCon

Europe 2026

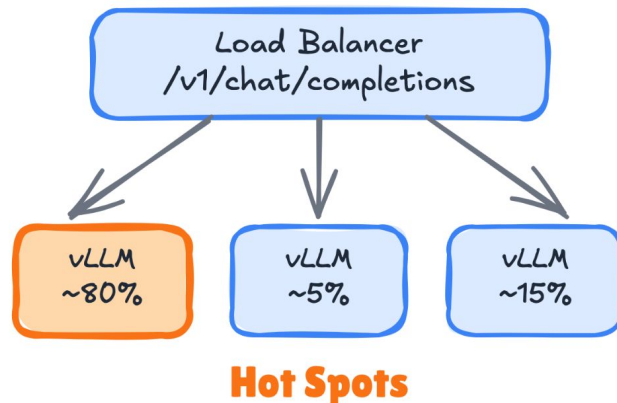
Kubernetes routing

- Round-robin (kube-proxy)
- Topology-aware (same-zone affinity)
- Path-based routing
- Works best when request cost is similar



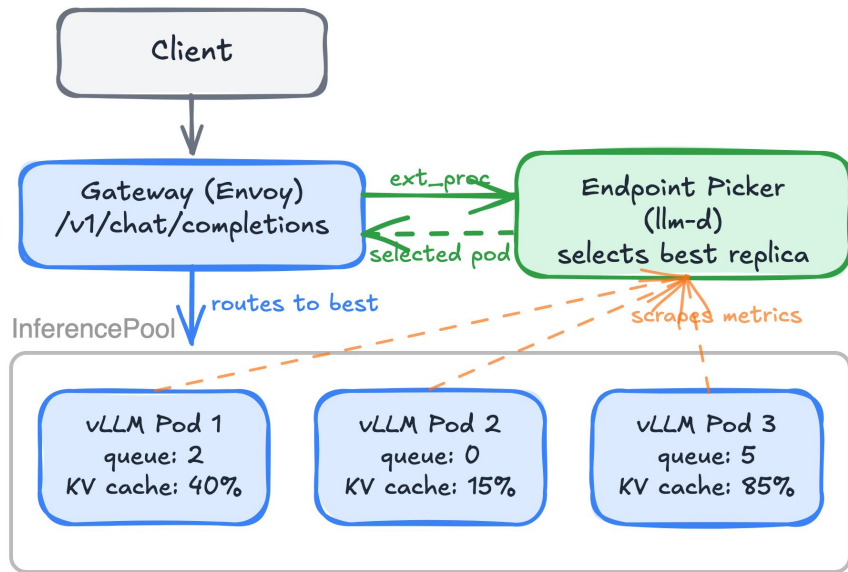
LLM routing breaks here

- All requests: identical POST /v1/chat/completions
- 10 tokens: 200 ms, GPU barely warm
- 4,000 tokens: 30 s, GPU fully occupied
- Reasoning model: 5+ min thinking time
- The router can't tell which is which



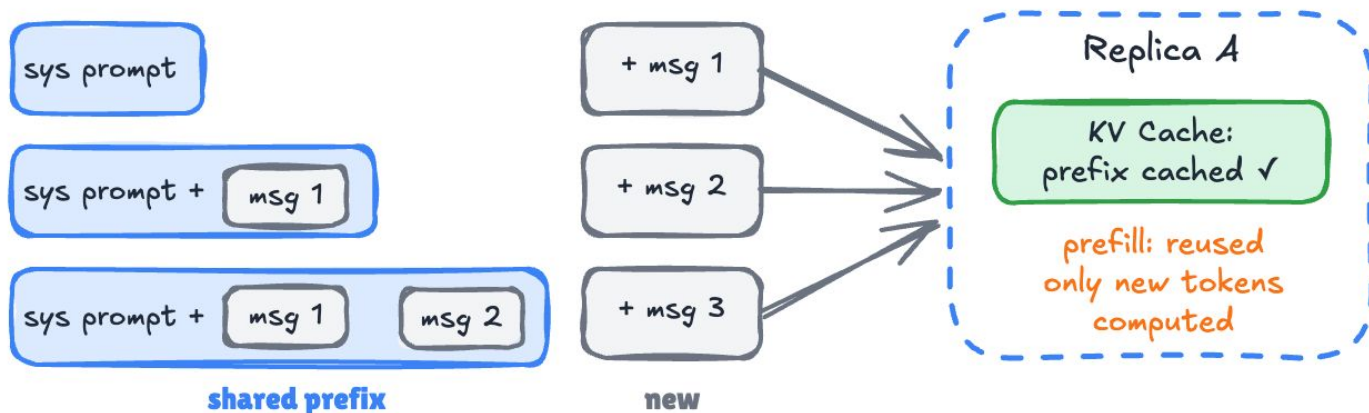
Route by Model State, Not Request Count

- **Gateway API Inference Extension**
(K8s native)
- **InferencePool**: group of pods, same model
- **Endpoint Picker**: selects best replica
- **Strategies**: capability(LoRA-aware), request priority, queue depth, prefix



Prefix Aware Routing

- LLMs have no memory between requests
- Full context (system prompt + history) sent every time
- Chatbots, agents, RAG: high prefix overlap
- Same prefix → same replica → KV cache reused
- KV cache already computed → skip redundant prefill



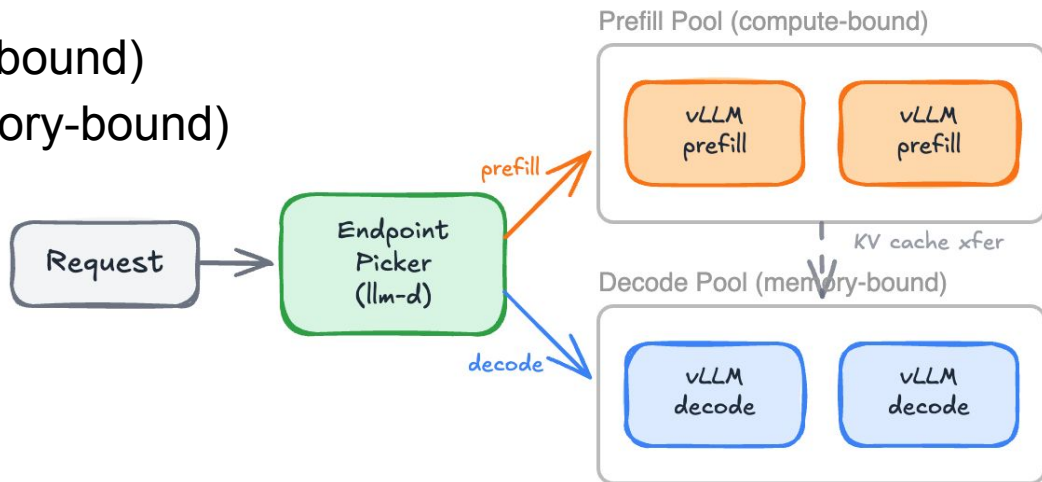
Disaggregated (prefill-decode) Routing

Inference phases

- Prefill: process input (compute-bound)
- Decode: generate output (memory-bound)

Routing optimization ladder

1. Only consider eligible replicas
2. Prefer cache presence
3. Be Load-aware
4. Tie-breaking policy



LLM routing is still an emerging area!

Pattern: RAG Composition



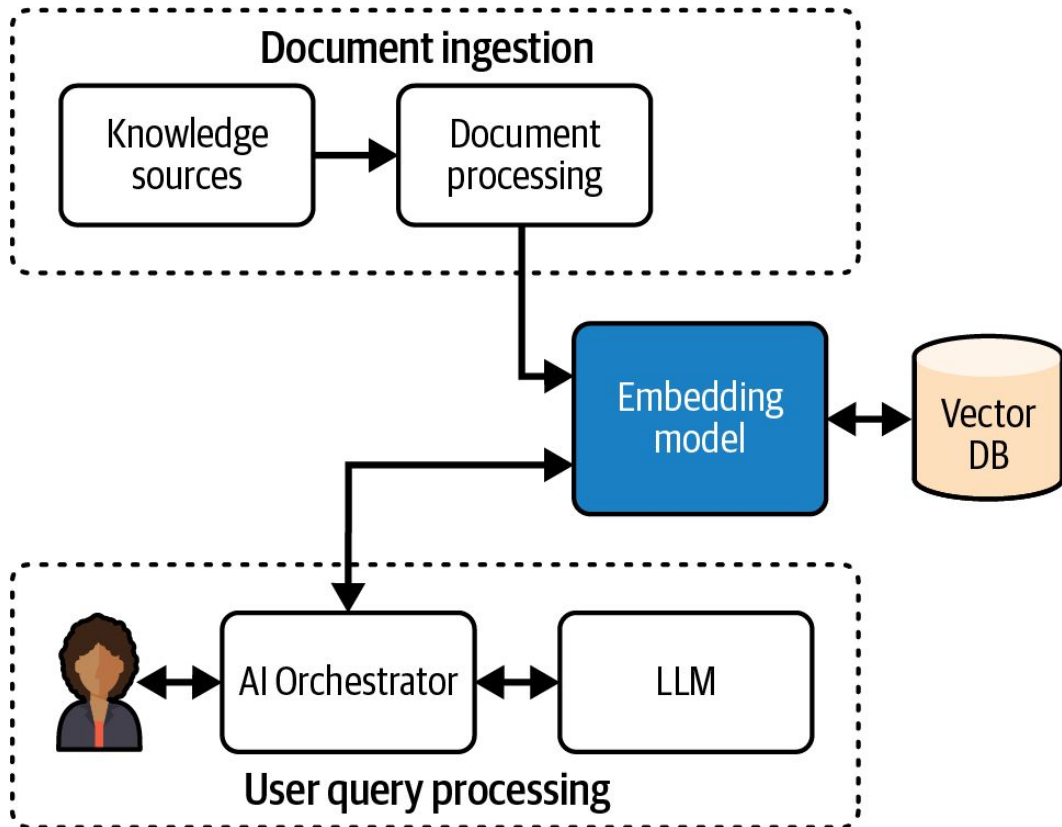
KubeCon



CloudNativeCon

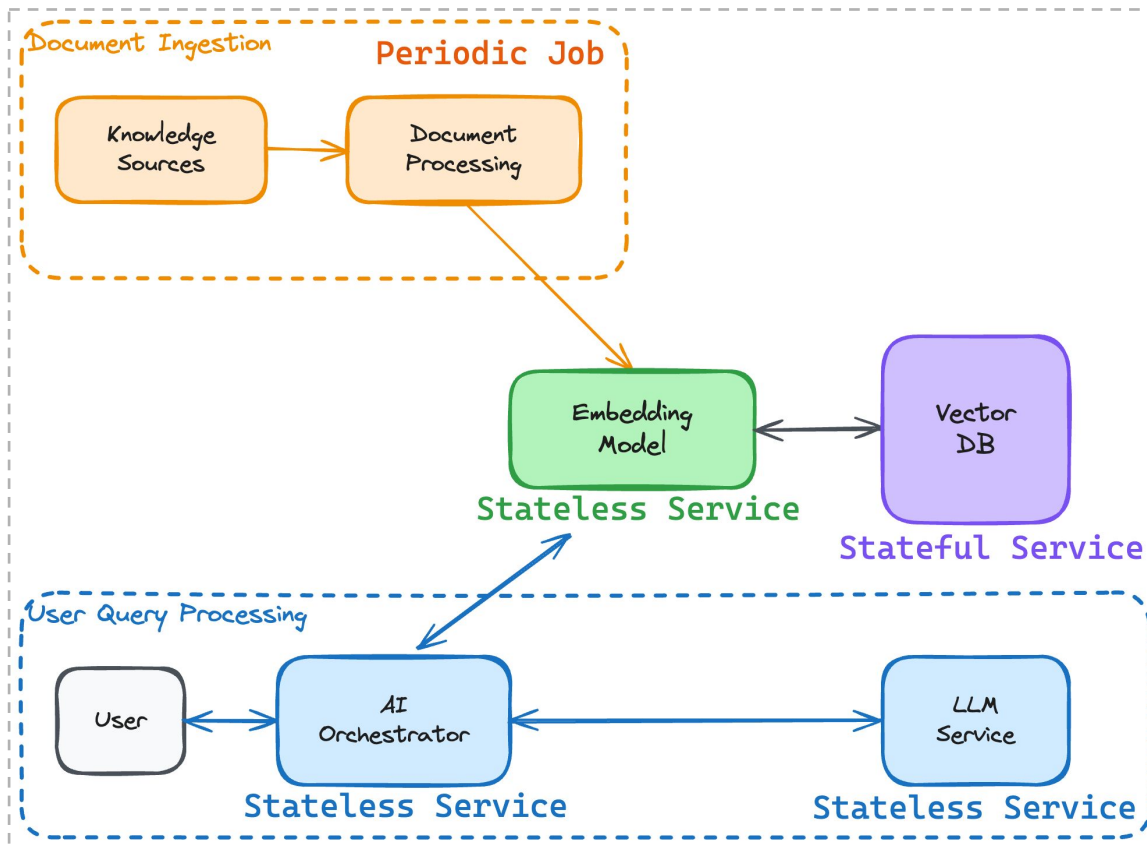
Europe 2026

- RAG Gives LLMs Your Domain Knowledge
- **Ingest:**
chunk docs → embed → vector store
- **Query:**
embed question → retrieve → augment prompt
- Same embedding model for both paths



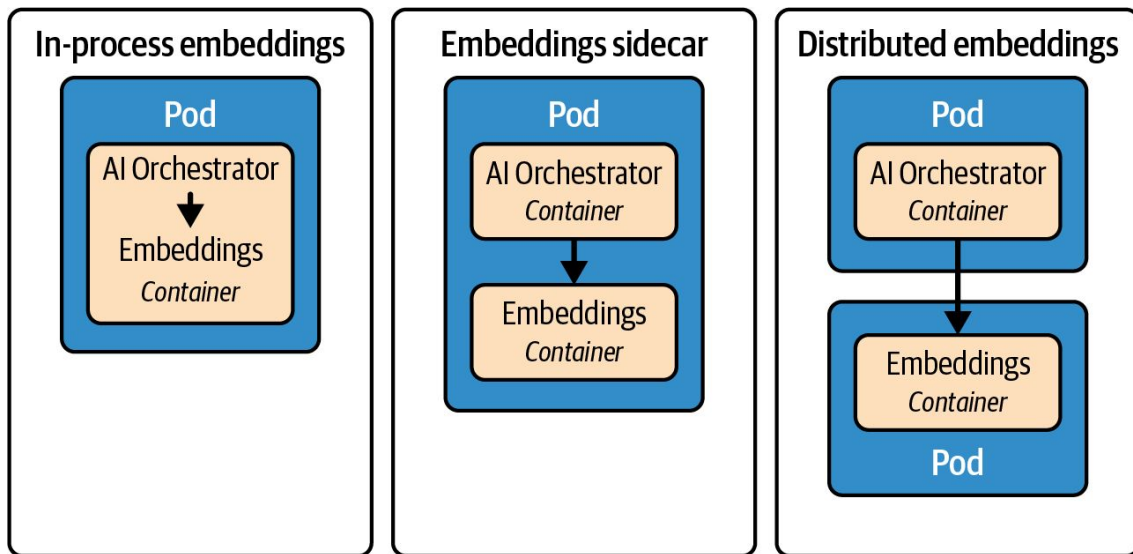
Every RAG Component Maps to a K8s Pattern

Orchestrator	Deployment
LLM Service	Deployment + GPU
Vector DB	StatefulSet + PVC
Embedding	Deployment
Ingestion	CronJob



Three Ways to Deploy Embeddings

In-process	Stateless Service	Simplest, but couples scaling
Sidecar	Sidecar	Shared lifecycle, localhost, independent GPU
Distributed	Stateless Service	Independent scaling, shared by query + ingestion





KubeCon



CloudNativeCon

Europe 2026

Questions ?

